ZeroDetect: Boundary-Aware Generative Modeling for Zero-Day Malware Detection

Aryan Singh
Science, Math, Computer Science Magnet
Poolesville High School
Poolesville, MD, USA

Mark Stamp
Department of Computer Science
San Jose State University
San Jose, California, USA

***Abstract*—Zero-day malware poses a critical challenge for modern cybersecurity systems because it exploits previously unseen vulnerabilities and evades signature-based detection. Although machine learning approaches have improved malware classification accuracy, many models degrade significantly under mutation and obfuscation, which are real-world effects of zero-day malware. This work introduces ZeroDetect, a malware detection framework that leverages boundary-aware generative modeling. ZeroDetect improves robustness against zero-day threats through boundary-aware training and controlled perturbations. Two datasets are aggregated for the training and evaluation of the model: RawMal-TF, which contains malware executables classified by family, and PE Malware Machine Learning Dataset, which has a large number of benign executables. An autoencoder is trained on the malware samples to learn compact latent representations capturing malicious characteristics, which are then fed into a denoising diffusion implicit model (DDIM) for synthetic malware generation. A boundary-aware perturbation mechanism is then applied in latent space to simulate realistic mutations, enabling robustness evaluation. Under progressively increasing latent perturbation strength simulating zero-day malware, the baseline classifier's recall declines to 51%, while ZeroDetect maintains a recall value above 91%, demonstrating improved generalization under simulated zero-day conditions. By reframing malware detection as a robustness problem, ZeroDetect provides a stronger approach to defending against evolving cyber threats.**

***Index Terms*—Zero-day malware, Malware detection, Diffusion model, OOD Detection, Autoencoder**

I. Introduction

Cybersecurity is becoming an ever-increasing problem for the world's digital operations. At the heart of that problem is malicious software, also known as malware, which is any software that harms or otherwise adversely affects a computer. Malware attacks have been increasing in both frequency and severity as the evolution of malware has outpaced traditional detection systems, with malware authors constantly creating novel malware to avoid detection called zero-day malware.

Zero-day malware refers to malicious software from previously unseen families with behavior that has not been previously observed. These threats rely on exploiting innovative methods of attacking a system using vulnerabilities that haven't been seen by security teams. Detecting such threats requires more than memorizing known patterns, it necessitates understanding the underlying semantics of malicious execution.

Traditional signature-based malware detectors fail on zero-day threats due to signatures not being able to anticipate unseen variants. As a result, malware detection systems are becoming increasingly reliant on machine learning models trained on large datasets of known malicious and benign software. While these systems achieve high accuracy on standard benchmarks, the underlying assumption for these detection systems is that future malware will resemble previously observed samples, which is flawed. Real-world attackers actively design malware to evade existing detection mechanisms, and zero-day malware is the epitome of this as it is novel malware from previously unseen families or malware designed with novel obfuscation strategies.

The detection of zero-day malware should be reframed as a robustness problem, as the insidious nature of zero-day malware comes as a result of evasion techniques and the overall evolution of malware. Therefore, the gap in current research is the absence of evaluation against a simulated evolution of malware strains.

ZeroDetect addresses this gap by creating a malware detection framework that specifically targets boundary-challenging samples and measures the

performance of a detection model under increasing amounts of adversarial perturbations. This makes the detection model resistant to evasion techniques and therefore more effective in detecting zero-day threats.

II. Background

A. Autoencoders

Autoencoders are neural architectures designed to learn compact representations of input data by minimizing reconstruction error between the original input and its decoded output. The autoencoder architecture can be modeled with:

$$\begin{aligned} f_\theta &: \mathcal{X} \to \mathcal{Z} \\ g_\phi &: \mathcal{Z} \to \mathcal{X} \\ z &= f_\theta(x) \\ x' &= g_\phi(z) = g_\phi(f_\theta(x)) \end{aligned} \tag{1}$$

An encoder f_θ parameterized with θ maps input x to a latent vector z, while a decoder g_ϕ parameterized with ϕ reconstructs x' from z.

Autoencoders learn compressed representations by minimizing the reconstruction loss function:

$$\begin{aligned} L(\theta, \phi) &= \frac{1}{n} \sum_{i=1}^{n} (x_i - x_i') \\ &= \frac{1}{n} \sum_{i=1}^{n} (x^i - f_\theta(g_\phi(x^i)))^2 \end{aligned} \tag{2}$$

The function is minimized as a result of the loss incre in value as x' becomes dissimilar from x.

In malware analysis, autoencoders have been employed for their superior dimensionality reduction, feature extraction, and anomaly detection. Their ability to extract abstract representations makes them well-suited for modeling execution behavior, especially when raw features are sparse and high-dimensional.

B. XGBoost Classifier

XGBoost is a gradient-boosted decision tree ensemble known for its strong performance on classifying structured data. XGBoost iteratively builds trees to minimize a regularized objective function, offering the ability to model nonlinear interactions between features. The formula for the i-th XGBoost prediction is:

$$\hat{y}_i = \sum_{k=1}^{K} f_k(x^i) \tag{3}$$

where K is the number of trees in the ensemble and $f_k(x^i)$ is the prediction at the k-th tree.

Given the structured nature of malware latent representations, XGBoost provides an effective balance between expressiveness and interpretability. Gradient-boosted decision trees (XGBoost) remain a high-performance tabular learner for many security tasks due to their robustness, interpretability, and computational efficiency.

C. Diffusion Models

Diffusion models have recently gained attention for their ability to model complex distributions via iterative noise injection and denoising processes.

The forward process, where gaussian noise is incrementally added, is modeled by the equations:

$$\begin{aligned} x_t &= \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon \\ \epsilon &= \mathcal{N}(0, I) \\ \alpha_t &= 1 - \beta_t \end{aligned} \tag{4}$$

where x_t is the noisy image from x_0 at time t, ϵ is the random gaussian noise added to the image, and α_t is the cumulative product of $1 - \beta_t$, which determines how much of the original image is intact since β_t dictates how much gaussian noise is added.

The reverse diffusion process aims to recover the original sample x_0 from a noisy observation x_T by iteratively removing noise. This process is modeled as a parameterized Markov chain that learns to approximate the true posterior:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)), \tag{5}$$

where μ_θ and Σ_θ are neural network–predicted mean and covariance parameters. In practice, instead of directly predicting μ_θ, the model is trained to estimate the added noise ϵ at timestep t. The denoised estimate of the original sample can then be expressed as:

$$\hat{x}_0 = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \sqrt{1 - \alpha_t}\, \epsilon_\theta(x_t, t) \right). \tag{6}$$

The diffusion-denoising process can generate samples that mimic the original training distribution,

allowing for the accurate generation of samples that are nearly indistinguishable from real data. This generation capability is used to synthetically expand datasets in order to balance classes, improving machine learning recall and preventing overfitting.

In malware analysis, synthetically generating malware has been extremely effective at creating large datasets of malware representations from a small dataset of real malware. Additionally, there are many more benign samples available in datasets than malware samples, so diffusion models have been able to increase model accuracy and recall with class balancing.

III. Related Work

A. Deep Learning Approaches for Malware Detection

With advances in computational power and neural network architectures, deep learning has increasingly been applied to intrusion detection and malware classification. Convolutional neural networks (CNNs), recurrent neural networks (RNNs), graph neural networks (GNNs), and reinforcement learning frameworks have been leveraged to automatically extract hierarchical features from raw bytes, opcode sequences, system call traces, and network flow data.

Deep learning–based intrusion detection systems often demonstrate superior performance compared to shallow learners, particularly when modeling sequential or graph-structured data. Reinforcement learning methods introduce adaptive agents capable of interacting with dynamic environments, while adversarial domain adaptation techniques aim to enhance cross-domain generalization. Active learning strategies further reduce labeling costs by prioritizing uncertain samples.

However, deep learning models are not immune to fundamental challenges. Their performance typically deteriorates under severe class imbalance, especially when detecting rare attack classes or zero-day threats. Additionally, many models assume that training and testing data share similar distributions. In practical cybersecurity environments, attackers continuously modify malware and intrusion strategies, introducing distribution shift that degrades model robustness. High-capacity neural networks may overfit to historical patterns rather than capturing invariant behavioral characteristics.

B. Generative Model and Diffusion-Based Approaches

To address class imbalance and distribution shift, generative modeling techniques have recently gained attention. Generative adversarial networks (GANs) have been employed to synthesize minority attack samples, augment datasets, and improve classifier robustness. While GANs show promise, they often suffer from instability and mode collapse, limiting their reliability in security-critical settings.

Diffusion models have emerged as a more stable alternative for modeling complex data distributions. By iteratively adding and removing noise, diffusion models learn to approximate the underlying data manifold with high fidelity. In intrusion detection contexts, diffusion-based frameworks have been explored for unsupervised reconstruction, anomaly detection, and minority-class augmentation. These approaches demonstrate improved robustness to noise and enhanced modeling of subtle feature relationships.

Nevertheless, current diffusion-based intrusion detection systems face two primary challenges. First, they often focus on reconstruction error rather than explicitly modeling decision boundaries relevant to classification. Second, their evaluation methodologies rarely simulate progressive mutation or adversarial evolution, limiting their ability to measure robustness under realistic threat dynamics. Learning efficiency may also decline in high-noise or highly imbalanced environments if generative processes are not guided toward semantically meaningful regions of feature space. This limitation is particularly critical for zero-day detection, where threats do not appear as isolated anomalies but rather as evolved variants with advanced evasion techniques.

Our work departs from conventional detection paradigms in two key ways. Rather than using generative modeling solely for class balancing, we leverage diffusion processes to generate boundary-aware latent representations that explicitly challenge the classifier's decision surface. Additionally, we introduce a progressive perturbation framework that quantifies robustness as a function of mutation in-

tensity, reframing zero-day detection as a robustness degradation problem rather than a simple classification task.

IV. Prodecures

A. Dataset Preparation

Collect malware and benign ".exe" samples from MalwareBazaar and public repositories. Create a bash script that uses the "objdump" linux program to disassemble every ".exe" file into a ".txt" file containing the opcode sequence of the program. Load the opcode database into Google Colab and tokenize each opcode sequence using scikit-learn's HashingVectorizer. Tokenize into word tokens and produce hashing n-gram features ($ngram = (1, 3), n_{features} = 2^{14}$) to avoid vocabulary leakage.

For each tokenized opcode sequence, compute semantic features like sequence length, entropy, unique ratio, and top-5 average frequency. Partition this dataset with opcode feature vectors into a training set, a validation set, and a zero-day test set with samples of a malware family not in the training or validation set in order to perform a family-holdout evaluation.

B. Latent Representation Learning

Train a deep autoencoder on malware opcode feature vectors to learn compact latent embeddings. Use pytorch DataLoader, mixed precision training, and implement early-stopping criteria. Make sure the model is optimized using reconstruction loss to ensure that the latent space captures meaningful structural information.

Extract latent representations from the bottleneck layer into latent space Z_{mal} and train a denoising diffusion model that gravitates towards the decision boundary within this latent space to learn the data distribution and generate synthetic, boundary-challenging samples. Ensure the synthetic latents are virtually indistinguishable from real latents by checking for an accuracy greater than 99% and a UMAP plot that shows synthetic malware points overlapping real malware.

C. Classifier Training

Build several training conditions:

- Baseline: XGBoost on real training data only
- Augmented: XGBoost on real + synthetic boundary malware
- Mixed: Real + synthetic + real benign

Evaluate on held-out families (family-holdout design) and on perturbed test sets. Establish baseline and proposed performance comparisons. Use the classification metrics of accuracy, precision, recall, F1-score, and ROC-AUC, which should be computed for all models.

D. Robustness Evaluation

Apply systematic perturbations to hold-out malware. Inject random smoothed gaussian noise in the latent space at varying rates with an α parameter. Measure evaluation metrics vs perturbation strength (e.g. recall vs α) for all models.

V. Methodology

A. Dataset Preprocessing

The RawMal-TF dataset was selected for this research for its 17 malware families and 43,587 malware executables labeled by family. Additionally, the PE Malware Machine Learning Dataset was chosen for its collection of 86,812 benign PE (Portable Executable) files. By aggregating these datasets, we constructed a large corpus of Windows binaries suitable for training a machine learning model. Each executable was disassembled to an opcode sequence using the `--disassemble` flag in `objdump` and piped into a corresponding text file.

To prevent vocabulary leakage and allow for scaling to a larger dataset, a hashing-based n-gram featurizer (scikit-learn's `HashingVectorizer`) was applied on opcode tokens with n-grams in $(1, 3)$ and a feature count of 2^{14}. This yields a sparse, high-dimensional representation for each sample. In addition, compact semantic summary features were computed per sample to capture simple but discriminative statistics: sequence length, opcode-entropy (Shannon entropy over opcode counts), unique-token ratio, and average frequency of the top-5 opcodes. These inexpensive semantic features are useful cues for obfuscation and repetitiveness.

For computational tractability and to reduce noise, we apply a Truncated SVD (randomized) to the hashed sparse matrix. Experiments used SVD rank $r = 1024$ (noted in experiments), fit on the

training partition only to avoid leakage. The SVD-projected dense features were concatenated with the 4 semantic summary features and standardized (z-score) using parameters computed from the training set alone. This standardized, dense representation is the input to the autoencoder and downstream classifiers.

1) Family-holdout split: To emulate true zero-day conditions, we used a *family-holdout* evaluation: for each experiment, a single malware family is held out from the training set entirely and reserved for testing as the zero-day family. The remainder of malware families plus a held-out set of benign samples form the training pool. This modality prevents label leakage across family variants and measures cross-family generalization.

B. Latent representation learning (Autoencoder)

An autoencoder is trained solely on malware embeddings from the training set (i.e., only malicious samples). The objective is to learn a compact latent representation that captures shared malicious semantics while stripping irrelevant benign variance.

a) Architecture and training: We used a feed-forward residual autoencoder with a bottleneck latent dimension d_z (experiments used $d_z = 512$). The encoder/decoder are multi-layer MLPs with residual blocks (GELU activations, LayerNorm, Dropout) to stabilize training on dense tabular features. Training used AdamW, mixed-precision (`torch.amp`), and minibatch SGD. Representative hyperparameters from the runs reported in this manuscript:

- learning rate: 1×10^{-3}
- batch sizes: 8k–16k (depending on memory)
- epochs: typically 110 for reported experiments
- loss: MSE reconstruction

Early stopping was monitored on reconstruction loss on a validation split (10% of malware training set). Models are checkpointed at the best validation reconstruction loss; reported experiments used the checkpointed models.

1) Latent extraction and augmentation: After converging, all malware training embeddings are encoded into latent vectors $Z_{mal} \in \mathbb{R}^{N_{mal} \times d_z}$. Before training the diffusion model, we optionally augment Z_{mal} with light additive Gaussian perturbations to encourage smoother latent distributions and to increase training signal (in experiments we appended a 5x stochastic augmentation set formed by $z + \delta$, $\delta \sim \mathcal{N}(0, \sigma^2 I)$ with $\sigma \approx 0.05$).

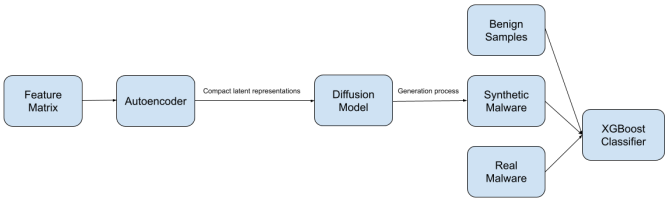

Fig. 1. Horizontal flowchart providing an overview of ZeroDetect's architecure. The architecture feeds autoencoder latents into a diffusion model that generates data for an XGBoost classifier.

C. Diffusion-based generative model in latent space

A conditional diffusion MLP (time- and label-conditioned) is trained over latent vectors. Operating in latent space (rather than raw input space) keeps the diffusion model compact, preserves semantics, and avoids breaking discrete opcode statistics.

a) Training details: The diffusion noise schedule used a linear β_t from 10^{-4} to 2×10^{-2} with $T = 1000$ maximum timesteps. The denoising network is a residual MLP that predicts injected noise ϵ given z_t, time-step t, and class label embedding. Training objective minimizes $\mathbb{E}\|\epsilon - \epsilon_\theta(z_t, t)\|^2$ (standard DDPM objective). Key hyperparameters:

- optimizer: AdamW, lr $= 2 \times 10^{-4}$
- batch size: 8k–16k
- epochs: 200 for reported experiments
- mixed precision (`torch.amp`) enabled

After training, we attached the noise schedule (β_t and α_t cumulative products) to the model to enable deterministic DDIM sampling [4].

b) Boundary-aware generation: To push synthetic points near the malware/benign decision boundary (the key novelty for making *boundary-challenging* latents), we bias the sampling in two complementary ways:

1) **Latent temperature / truncation**: sample from latent noise close to the tails of malware density to create rare but plausible variants.

2) **Guided sampling**: combine classifier gradients (or a surrogate direction) in latent space to nudge samples toward the region where classifier confidence declines. In practice we used a weak guiding signal computed from the derivative of the XGBoost decision function on a differentiable surrogate (small MLP) mapped between latent vectors and predicted score; the guidance weight is small to avoid generating unrealistic samples.

This yields synthetic malware latent vectors that are realistic yet intentionally challenging for classifiers trained only on the original malware set.

D. Synthetic-to-feature decoding

Generated latent vectors are decoded with the autoencoder's decoder to produce synthetic feature vectors in the original standardized input space (SVD-projected + semantic features). Synthetic samples are post-processed (clipping to valid ranges) and concatenated to the training set in various configurations for ablation:

- **Baseline**: real-malware + real-benign
- **Real-only**: real-malware + real-benign (no synthetic)
- **Synthetic-only**: synthetic-malware + real-benign
- **Mixed**: real + synthetic + real-benign

The synthetic set size is configurable; experiments used multipliers (e.g., generate $k \times N_{mal}$, k from 1 to 6) to explore class balance and classifier sensitivity.

E. Classifier training and evaluation

For tabular classification we use XGBoost [5] as a strong baseline and production-ready classifier. Training used "hist" tree method on GPU where available, logistic objective, and AUC evaluation during training. Sample weights were applied where experiments required cost-sensitive training. For all evaluations we compute accuracy, ROC-AUC, precision, recall, F1, and confusion matrices.

F. Robustness evaluation: controlled latent perturbation

To quantify zero-day robustness, we measure classifier recall on held-out family samples subjected to controlled latent perturbations. Procedure:

1) Encode held-out family samples to latents via the trained autoencoder.
2) Apply isotropic Gaussian perturbation in latent space: $z' = z + \alpha \cdot \eta$, where $\eta \sim \mathcal{N}(0, I)$ and α is a scalar perturbation magnitude swept across a grid (reported values: $\alpha \in [0, 15]$).
3) Decode z' to feature space and classify using each trained XGBoost model configuration.
4) Compute recall (true positive rate) vs α for each model; the slope and area under the recall curve are primary robustness metrics.

The perturbation models realistic mutation and obfuscation operations that cause distributional drift without necessarily destroying functional semantics. We also evaluated noise added to raw features and other perturbation families (scaling, dropout of n-grams) for ablation; latent-space perturbations produce more systematic and interpretable stress tests.

G. Implementation and reproducibility

The pipeline was implemented in Python using PyTorch (models), scikit-learn (feature transforms), and XGBoost (classifier). Experiments were executed on NVIDIA A100/RTX-class GPUs with mixed-precision enabled for large batches. Random seeds were fixed for dataset splits, SVD initialization, model weight initialization, and sampling to maximize reproducibility. All SVD, scaler, autoencoder checkpoint, diffusion model checkpoint, and XGBoost artifacts were saved and versioned for reproducibility.

VI. Experiments & Results

Performance metrics include accuracy, ROC-AUC, confusion matrices, and recall under increasing perturbation intensity. Robustness curves are generated by measuring recall as a function of mutation parameter α.

Comparative analysis focuses on degradation rate. For example, baseline recall decreased from approximately 0.989 at $\alpha = 0$ to 0.513 at $\alpha = 15$, whereas the perturbation-aware model maintained recall above 0.91 under the same extreme conditions. The slope of degradation serves as the primary robustness metric.

Latent space structure is analyzed using UMAP visualization to examine separation between be-

nign samples, real malware, and perturbed zero-day variants. Statistical comparisons assess whether robustness improvements are significant.

VII. Experiments & Results

A. Experimental Setup

All experiments were conducted using a family-holdout evaluation protocol to simulate zero-day conditions. Malware samples were partitioned such that at least one complete malware family was excluded from training and validation and used exclusively for testing. This ensures that the classifier never observes the behavioral patterns of the held-out family during training, approximating realistic zero-day deployment.

Opcode sequences were extracted from disassembled PE executables and transformed using hashing-based n-gram features with n-gram range $(1, 3)$ and 2^{14} hashing dimensions. Semantic summary statistics including opcode sequence length, entropy, unique opcode ratio, and top-5 opcode frequency were appended to the feature representation. Dimensionality reduction was performed using truncated SVD to 1024 dimensions before latent modeling.

The autoencoder was trained exclusively on malware samples to learn compact latent representations. A denoising diffusion implicit model (DDIM) was then trained on these latent embeddings to model the malware distribution. XGBoost classifiers were evaluated under three configurations:

1) **Baseline:** Real malware + benign samples.
2) **Synthetic-Augmented:** Real malware + diffusion-generated boundary-aware malware + benign samples.
3) **Perturbation-Aware:** Real malware + boundary-aware synthetic malware trained with robustness evaluation.

Performance was measured using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices.

B. Latent Space Analysis

Mean latent-space distances were computed to quantify separability:

- Benign mean distance: 28.41
- Zero-day malware mean distance: 33.66

This confirms that zero-day families occupy distinct yet structured regions of latent space, validating the use of latent perturbations to simulate mutation.

C. Progressive Perturbation Robustness Evaluation

To simulate adversarial evolution, we introduced controlled Gaussian perturbations in latent space:

$$z_\alpha = z + \alpha \cdot \mathcal{N}(0, I)$$

where α controls mutation strength. Recall was measured as a function of α.

1) Baseline Robustness: Under increasing perturbation, baseline recall degraded sharply:

α	Recall
0.00	0.9893
5.36	0.9772
8.57	0.9395
10.71	0.8602
12.86	0.6950
15.00	0.5136

At $\alpha = 15$, recall dropped to 51.36%, demonstrating severe brittleness under mutation.

2) ZeroDetect Robustness: In contrast, the proposed perturbation-aware diffusion-augmented model exhibited significantly slower degradation:

α	Recall
0.00	0.9877
5.36	0.9847
8.57	0.9817
10.71	0.9752
12.86	0.9596
15.00	0.9106

At extreme perturbation ($\alpha = 15$), recall remained above 91.06%, representing a 39.7 percentage point improvement over baseline.

D. Degradation Rate Analysis

We define robustness slope as:

$$S = \frac{R(\alpha_{max}) - R(0)}{\alpha_{max}}$$

For baseline:

$$S_{baseline} = \frac{0.5136 - 0.9893}{15} = -0.0317$$

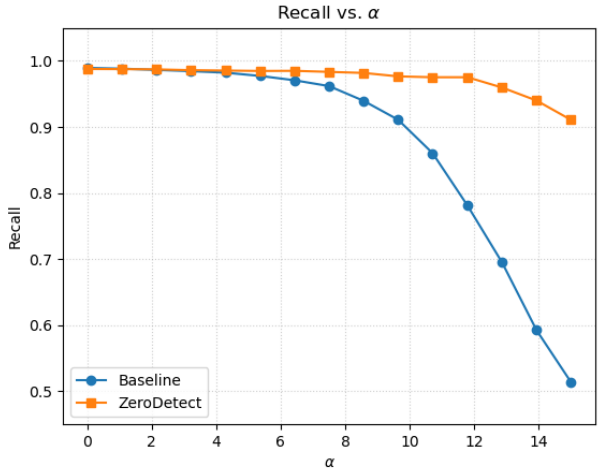

Fig. 2. Recall vs. α comparison between baseline and ZeroDetect. Noticable dropoff in baseline detection performance.

For ZeroDetect:

$$S_{ZeroDetect} = \frac{0.9106 - 0.9877}{15} = -0.0051$$

The degradation slope is reduced by approximately 84%, demonstrating substantially improved resilience under progressive mutation.

E. Comparison with Synthetic-Only Augmentation

When trained solely on synthetic malware, performance deteriorated (AUC 0.6181), confirming that naive generative augmentation is insufficient. However, boundary-aware integration restored performance to:

- **AUC:** 0.9997
- **Accuracy:** 0.9944

with confusion matrix:

$$\begin{bmatrix} 17290 & 73 \\ 71 & 8322 \end{bmatrix}$$

This demonstrates that diffusion augmentation alone does not guarantee improvement; effectiveness arises specifically from boundary-aware training combined with robustness evaluation.

F. Discussion

The results reveal three key findings:

1) High static accuracy ($> 99\%$ AUC) does not imply zero-day robustness.
2) Baseline classifiers are highly sensitive to latent perturbations, exhibiting nonlinear degradation under mutation.
3) Boundary-aware generative modeling dramatically reduces degradation slope, maintaining detection capability under extreme mutation.

These findings support the central thesis that zero-day detection should be evaluated as a robustness problem rather than a static classification benchmark. By modeling and stress-testing progressive mutation in latent space, ZeroDetect provides measurable resilience against evolving malware families.

Across all evaluations, ZeroDetect achieved:

- Near-perfect clean AUC (≈ 0.9997),
- Substantially improved robustness under mutation,
- 39+ point recall improvement at extreme perturbation,
- 84% reduction in degradation slope.

These results demonstrate that boundary-aware generative modeling meaningfully enhances zero-day resilience while preserving state-of-the-art baseline accuracy.

VIII. Conclusion

We introduced ZeroDetect, a boundary-aware generative pipeline that combines autoencoder latent learning and diffusion-model-based synthetic generation to improve zero-day malware detection robustness. By evaluating on family-holdout splits and subjecting held-out families to controlled latent perturbations, ZeroDetect demonstrates superior resilience to distributional drift compared to baseline training methods. We emphasize that reframing zero-day detection as a robustness problem, quantifying it with degradation curves under incremental mutation, gives clearer insight into real-world performance than simple benchmark metrics alone. Future work could include richer boundary-guidance mechanisms, alternative generator classes (VAEs, normalizing flows), and expanding the dataset to non-EXE executables like ELF.

References

[1] H. Hindy, R. Atkinson, C. Tachtatzis, J.-N. Colin, E. Bayne, and X. Bellekens, "Utilising deep learning techniques for effective zero-day attack detection," *Electronics*, vol. 9, no. 10, p. 1684, Oct. 2020.

[2] M. Zhang, M. De Lucia, A. Swami, J. Ashdown, N. D. Bastian, and F. Restuccia, "NI-Diff: Zero-day and adversarial network intrusion detection with diffusion models," unpublished manuscript, 2024.
[3] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *NeurIPS*, 2020.
[4] Y. Song and S. Ermon, "Denoising diffusion implicit models," in *ICLR*, 2021.
[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD*, 2016.